

2pt 2pt

Correction — Réponses uniquement

Exercice 1

```
let entiers_a_partir_de_2 =  
  Flux.unfold (fun n -> Some (n, n + 1)) 2
```

Exercice 2

```
let rec crible f =  
  match Flux.uncons f with  
  | None -> Flux.vide  
  | Some (h, t) ->  
    Flux.cons h  
      (crible  
        (Flux.filter (fun n -> n mod h <> 0) t))
```

```
let premiers = crible entiers_a_partir_de_2
```

Exercice 3

```
type q0  
type q1  
type q2  
type q3  
type q4
```

```
type _ t
```

```
val debut : unit -> q0 t  
val chiffre : q0 t -> q1 t  
val chiffre : q1 t -> q2 t  
val chiffre : q2 t -> q3 t  
val chiffre : q3 t -> q4 t  
val reset : _ t -> q0 t  
val valid : q4 t -> bool
```

Exercice 4

```
type ('a,'b,'c,'d) t
```

```
val debut : unit -> (zero, zero, zero, zero) t  
val chiffre :  
  ('a,'b,'c,'d) t ->  
  'e chfr ->  
  ('b,'c,'d,'e) t  
val reset : _ t -> (zero, zero, zero, zero) t  
val valid : _ t -> bool
```

Exercice 5

```
type res =  
  | Done  
  | Yield of continuation  
  | Fork of program * continuation  
  | Sleep of float * continuation
```

Exercice 6

```
let sleep (s : float) : unit =  
  D.shift p (fun k -> Sleep (s, k))
```

Exercice 7

```
type proc = float * continuation  
  
| Sleep (s, k) ->  
  Queue.push (Sys.time () +. s, k) procs;  
  loop (Queue.pop procs ())  
  
| Done ->  
  if Queue.is_empty procs then ()  
  else  
    let (t, k) = Queue.pop procs in  
    if t <= Sys.time () then  
      loop (k ())  
    else (  
      Queue.push (t, k) procs;  
      loop Done  
    )
```